

Procedimento de Implementação: Proxy Reverso Nginx (meu-proxy:producao)

Documento: Procedimento de Implementação **Versão:** 1.1 **Data:** 27 de Outubro de 2025 **Elaborado por:** Ermirio Bonfim - Process Engineer **Revisado por:** [A ser preenchido pelo usuário]

1. Introdução

Este documento detalha o procedimento técnico para a implantação do serviço de **Proxy Reverso Nginx**, encapsulado na imagem Docker `meu-proxy:producao`. O foco é uma implantação em ambiente **offline** ou com restrições de rede, utilizando um pacote de implantação pré-preparado e orquestração via Docker Compose.

1.1. Objetivo

Garantir a correta instalação, configuração e inicialização do proxy reverso, que atua como ponto de entrada unificado para diversas aplicações internas (Frontends React, Backends Django/Flask, serviços de monitoramento e automação), conforme as regras de roteamento definidas no arquivo `nginx.conf` fornecido.

1.2. Escopo

O procedimento abrange desde a verificação de pré-requisitos até a inicialização e verificação do serviço em um ambiente baseado em Docker/Docker Compose, com comandos adaptados para ambientes Windows e Linux.

2. Pré-requisitos e Pacote de Implantação

2.1. Requisitos do Servidor

O servidor de destino (máquina offline) deve ter os seguintes softwares instalados e em execução:

Componente	Versão Mínima	Finalidade
Sistema Operacional	Linux (Recomendado) ou Windows/macOS	Hospedagem do Docker
Docker Engine	18.09+	Execução de containers
Docker Compose	1.25+ (ou <code>docker compose CLI</code>)	Orquestração e gerenciamento do container
Aplicações de Destino	Variável	Os serviços de destino (Backends, Frontends, Grafana, etc.) devem estar rodando e acessíveis via <code>host.docker.internal</code> nas portas especificadas.

2.2. Conteúdo do Pacote de Implantação

O pacote de implantação offline (`Proxy_Offline_Package`) deve conter os seguintes arquivos:

Arquivo	Descrição	Uso na Implantação
<code>proxy-image.tar</code>	Imagem Docker compactada (<code>meu-proxy:producao</code>).	Carregamento da imagem no repositório Docker local.
<code>docker-compose.yml</code>	Arquivo de orquestração do serviço.	Definição dos parâmetros de execução do container.
<code>nginx.conf</code>	Cópia da configuração do Nginx (apenas para referência).	Documentação das regras de roteamento.

3. Detalhes da Configuração do Proxy

A imagem `meu-proxy:producao` contém uma configuração Nginx que define o roteamento para múltiplos serviços internos. O proxy está configurado para ouvir na porta **80** do container, que é mapeada para a porta **3000** do host pelo `docker-compose.yml`.

3.1. Mapeamento de Portas e Redes

O arquivo `docker-compose.yml` define o mapeamento de portas e a comunicação com o host:

```
version: '3.8'
services:
  nginx-proxy:
    image: meu-proxy:producao
    container_name: 01_proxy_reverse
    ports:
      - "3000:80" # Host:3000 -> Container:80
    extra_hosts:
      - "host.docker.internal:host-gateway" # Permite acesso aos serviços do
host
    restart: always
```

3.2. Regras de Roteamento (Upstreams e Locations)

O arquivo de configuração do Nginx (internamente em `/etc/nginx/conf.d/default.conf`) define os seguintes *upstreams* (serviços de destino) e suas regras de roteamento (*locations*):

Serviço (Upstream)	Destino (Host Port)	Aplicações Roteadas
frontend_react_mis	host.docker.internal:3005	Rota principal (/) e fallback.
backend_django_mis	host.docker.internal:8000	Rotas de API (/api/ , /admin/ , /static/admin/ , /sincronizar-skus/).
frontend_react_density	host.docker.internal:3006	Rota /density-app/ (inclui suporte a WebSocket).
backend_flask_density	host.docker.internal:5001	Rota /density-api/ (com reescrita de URL).
frontend_react_predicao	host.docker.internal:3007	Rota /prediction-app/ (inclui suporte a WebSocket).
backend_flask_predicao	host.docker.internal:5004	Rota /predicao-api/ (com reescrita de URL).
grafana_service	host.docker.internal:3001	Rota /grafana/ (inclui suporte a WebSocket).
influxdb_service	host.docker.internal:8181	Rota /influxdb/ .
nodered_service	host.docker.internal:1880	Rota /nodered/ (inclui suporte a WebSocket).
nodered_embalagem	host.docker.internal:1884	Rota /nodered-embalagem/ (inclui suporte a WebSocket).
nodered_process_paking	host.docker.internal:1885	Rota /nodered-process-paking/ (inclui suporte a WebSocket).

Nota Técnica: O uso de `host.docker.internal` é crucial para que o container possa se comunicar com os serviços que estão rodando diretamente na máquina host, o que é comum em ambientes de desenvolvimento ou implantações monolíticas em máquinas únicas.

4. Procedimento de Implantação

Siga os passos abaixo no servidor de destino.

Passo 1: Acesso e Navegação ao Pacote

1. Acesse o servidor de destino.
2. Navegue até o diretório onde o pacote de implantação foi copiado:

Terminal Linux (Bash)	Terminal Windows (CMD/PowerShell)
<pre>cd /caminho/para/Proxy_Offline_Package</pre>	<pre>cd C:\caminho\para\Proxy_Offline_Package</pre>

Passo 2: Carregamento da Imagem Docker

A imagem do proxy deve ser carregada do arquivo `.tar` para o repositório local do Docker, pois não há acesso a um registro externo.

1. Execute o comando de carregamento:

Terminal Linux (Bash)	Terminal Windows (CMD/PowerShell)
<pre>docker load -i proxy-image.tar</pre>	<pre>docker load -i proxy-image.tar</pre>

1. **Verificação:** Confirme que a imagem `meu-proxy:producao` foi carregada com sucesso.

Terminal Linux (Bash)	Terminal Windows (CMD/PowerShell)
<pre>docker images grep meu-proxy</pre>	<pre>docker images findstr meu-proxy</pre>

Passo 3: Inicialização do Serviço

Utilize o Docker Compose para criar e iniciar o container em modo *detached* (segundo plano).

1. Execute o comando de inicialização:

Terminal Linux (Bash)	Terminal Windows (CMD/PowerShell)
<code>docker compose up -d</code>	<code>docker compose up -d</code>

1. O Docker Compose lerá o `docker-compose.yml` , criará o container `01_proxy_reverse` e o iniciará.

Passo 4: Verificação Pós-Implantação

1. **Verificar Status do Container:** Confirme que o container está em execução.

Terminal Linux (Bash)	Terminal Windows (CMD/PowerShell)
<code>docker ps grep 01_proxy_reverse</code>	<code>docker ps findstr 01_proxy_reverse</code>
<i>O status deve ser "Up".</i>	<i>O status deve ser "Up".</i>

1. **Verificar Logs:** Verifique os logs do Nginx para garantir que não houve erros de inicialização.

Terminal Linux (Bash)	Terminal Windows (CMD/PowerShell)
<code>docker compose logs</code>	<code>docker compose logs</code>

1. **Teste de Conectividade:** Acesse a URL do proxy no navegador para confirmar o roteamento.

- Se o acesso for feito localmente no host: `http://localhost:3000/`
- Se o acesso for feito remotamente: `http://<IP_DO_HOST>:3000/`

O acesso à rota principal (/) deve ser roteado para o `frontend_react_mis` na porta 3005 do host. Teste também uma rota específica, como `/density-app/` .

5. Manutenção e Troubleshooting

Problema	Causa Potencial	Solução
Container não inicia	Conflito de porta (3000 já em uso).	Pare o processo que está usando a porta 3000 ou edite o <code>docker-compose.yml</code> para usar outra porta do host (ex: <code>4000:80</code>).
"502 Bad Gateway"	O serviço de destino (<code>host.docker.internal:<PORTA></code>) não está acessível ou não está rodando.	Verifique se a aplicação de destino (ex: <code>frontend_react_mis</code> na porta 3005) está ativa no host.
"404 Not Found"	Rota não definida no <code>nginx.conf</code> ou erro de reescrita.	Verifique o <code>nginx.conf</code> (dentro do container) e as regras de <i>location</i> .
Problemas de WebSocket	Headers de conexão ausentes.	A configuração fornecida já inclui os headers necessários (<code>proxy_set_header Upgrade</code> , <code>proxy_set_header Connection</code> "upgrade"), mas verifique se foram sobrescritos.

5.1. Procedimento de Rollback

Em caso de falha crítica ou necessidade de reverter a implantação:

1. Parar e Remover o Container:

Terminal Linux (Bash)	Terminal Windows (CMD/PowerShell)
<code>docker compose down</code>	<code>docker compose down</code>

1. Remover a Imagem (Opcional):

Terminal Linux (Bash)	Terminal Windows (CMD/PowerShell)
<code>docker rmi meu-proxy:producao</code>	<code>docker rmi meu-proxy:producao</code>

1. **Restaurar Configuração Anterior:** Se houver uma versão anterior do proxy, siga o procedimento de implantação com os arquivos da versão anterior.
-

Anexo A: Configuração Nginx (Referência)

O arquivo `nginx.conf` (ou `default.conf` dentro do container) define os seguintes *upstreams* e *locations*:

```
# Define os "upstreams" (serviços de destino)
upstream frontend_react_mis      { server host.docker.internal:3005; }
upstream backend_django_mis     { server host.docker.internal:8000; }
# ... (demais upstreams)

server {
    listen 80;
    server_name localhost;

    # Exemplo de roteamento para Backend Django
    location /api/ {
        proxy_pass http://backend_django_mis;
        # ... (headers)
    }

    # Exemplo de roteamento para Frontend React (com WebSocket)
    location /density-app/ {
        proxy_pass http://frontend_react_density/;
        # ... (headers de WebSocket)
    }

    # Rota Genérica (Fallback)
    location / {
        proxy_pass http://frontend_react_mis;
        # ... (headers)
    }
}
```